


Università di Catania

DIPARTIMENTO DI MATEMATICA E INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA

*Comis Lorenzo - 1000084704*
*Sciacca Alessandro - 1000085546*

# Implementazione di un Classificatore per Sentiment Analysis tramite Reti Neurali

RELAZIONE PROGETTO MACHINE LEARNING

Anno Accademico 2024 – 2025

# Sommario

1 Introduzione al Problema ..... 3

2 Il Dataset ..... 3

2.1 Etichettatura e classi ..... 4

2.2 Statistiche testuali ..... 5

2.3 Organizzazione del dataset ..... 7

3 Metodi ..... 8

3.1 Pre-processing del testo ..... 8

3.2 Suddivisione del dataset ..... 8

3.3 Data augmentation e bilanciamento delle classi ..... 9

3.4 Costruzione della matrice TF-IDF ..... 9

3.5 Il modello neurale ..... 9

3.6 Addestramento del modello ..... 10

4 Valutazione del modello ..... 11

5 La Demo: una applicazione reale del modello ..... 13

5.1 Collegamenti con il notebook ..... 13

5.2 Funzionamento della Demo ..... 13

Conclusione ..... 15

Indice delle figure ..... 16

# 1 Introduzione al Problema

La **Sentiment Analysis** è uno dei task più famosi del Natural Language Processing (NLP) che ha l'obiettivo di determinare automaticamente l'opinione o l'emozione espressa in un testo. I testi vengono generalmente classificati in tre categorie principali: **positivo**, **neutrale** o **negativo**. Questa tecnica trova applicazione in numerosi ambiti, come sui social media, l'analisi delle recensioni di prodotti e nel servizio clienti automatizzato.

In questo progetto ci siamo occupati di costruire un sistema automatico di Sentiment Analysis applicato a testi (tweet) in lingua inglese, con l'obiettivo di addestrare un modello in grado di classificare nuovi testi in base al loro contenuto. Il lavoro è stato sviluppato partendo da un dataset etichettato, attraverso un processo completo che include:

- Pre-processing e pulizia dei dati testuali
- Bilanciamento delle classi tramite Data Augmentation
- Trasformazione dei testi in vettori numerici (TF-IDF)
- Addestramento di un classificatore basato su una rete neurale feed-forward
- Valutazione delle performance tramite metriche standard
- Realizzazione di una demo per testare nuove frasi inserite dall'utente

# 2 Il Dataset

Il dataset utilizzato per questo progetto contiene una raccolta di **tweet in lingua inglese**, ciascuno accompagnato da un'etichetta di **sentiment** che ne identifica il tono emotivo come **positivo**, **neutro** o **negativo**. I dati testuali, per loro natura, presentano una notevole variabilità in termini di lunghezza, lessico, punteggiatura ed elementi specifici del linguaggio social (es. menzioni, hashtag, URL).

In una prima fase esplorativa del progetto, è stato utilizzato un dataset molto ridotto (circa 1.000 tweet), con l'obiettivo di verificare il corretto funzionamento della pipeline. Tuttavia, l'accuratezza ottenuta risultava eccessivamente elevata e non realistica, sinonimo di **overfitting** e di una scarsa eterogeneità del campione. Per questa ragione, si è deciso di

passare a un dataset più ampio, contenente **oltre 25.000 tweet** per migliorare la rappresentatività e robustezza del modello anche in contesti reali.

## 2.1 Etichettatura e classi

Ogni tweet e stato etichettato secondo uno dei tre sentiment:

- **Positive** (etichetta numerica 2)
- **Neutral** (etichetta numerica 1)
- **Negative** (etichetta numerica 0

La distribuzione iniziale delle etichette nel dataset è risultata **sbilanciata**, con una predominanza della classe neutrale.

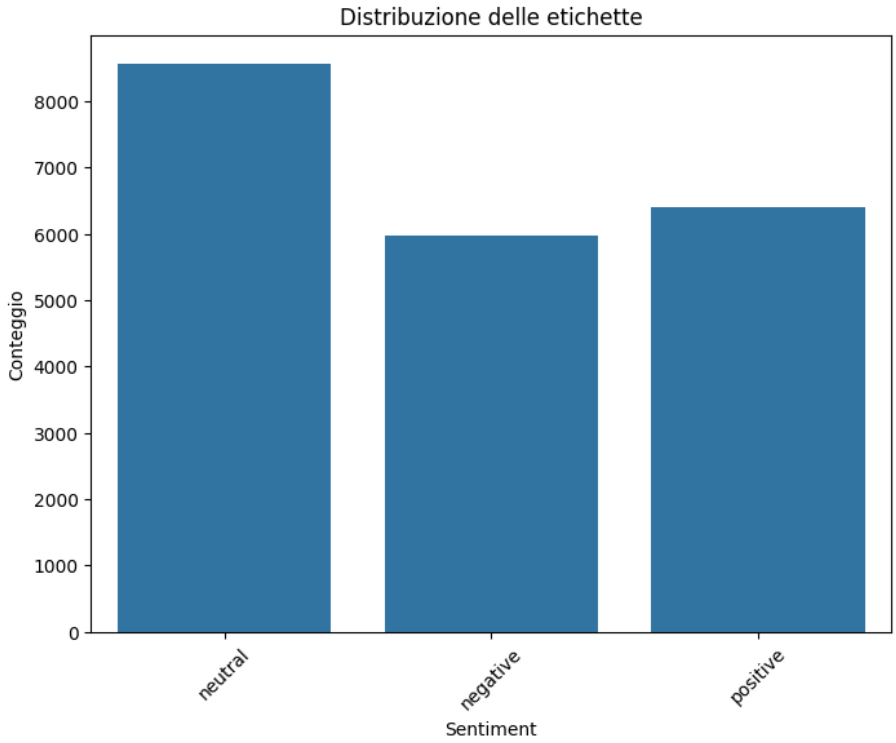



*Figura 1 - Distribuzione iniziale delle etichette nel dataset*

## 2.2 Statistiche testuali

È stata condotta un'analisi esplorativa della lunghezza media dei tweet suddivisi per classe, evidenziando le differenze nel numero delle parole utilizzate a seconda del tipo di sentimento espresso.

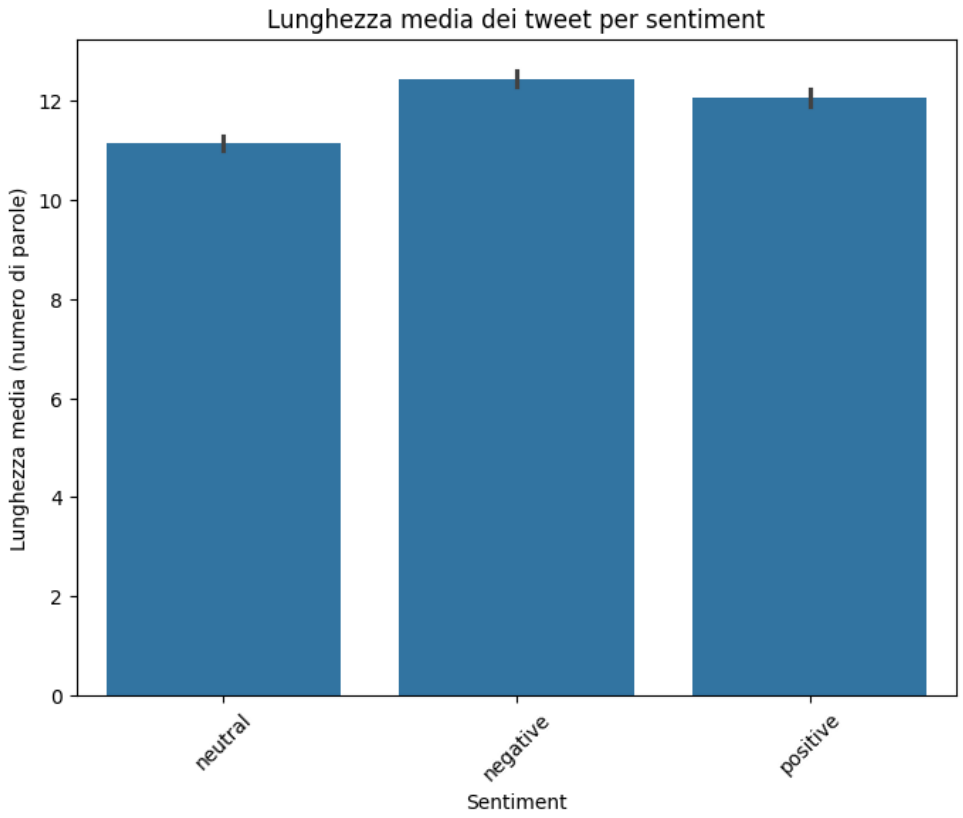



*Figura 2 - Lunghezza media dei tweet per classe*

Inoltre, per una migliore comprensione del contenuto testuale prevalente in ciascuna categoria, sono state generate delle **word cloud**, che rappresentano graficamente le parole più frequenti. Più una parola è grande e più è frequente per la propria etichetta.




*Figura 3 - Word Cloud classe Positive*

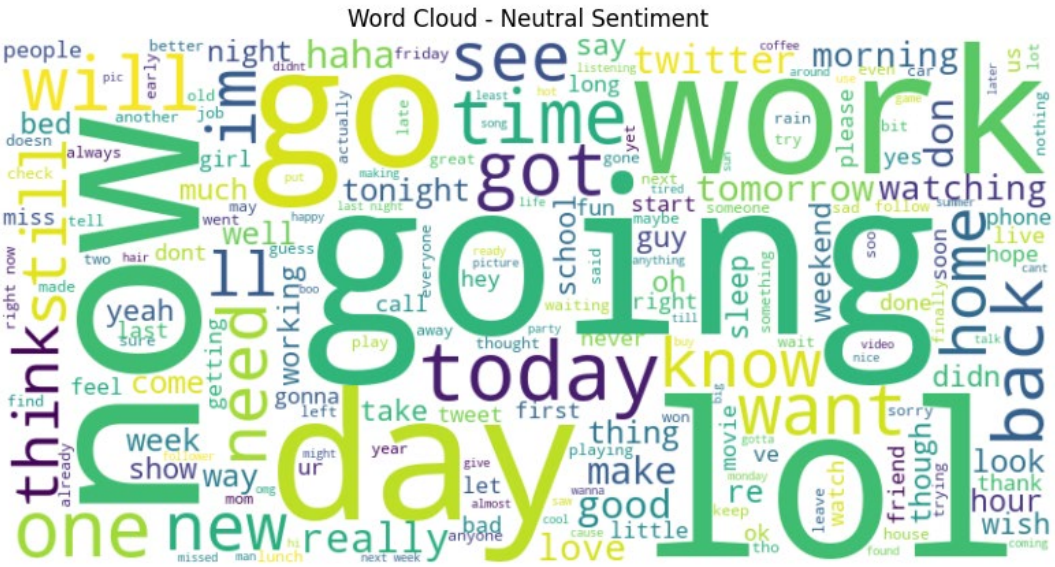



*Figura 4 - Word Cloud classe Neutral*




*Figura 5 - Word Cloud classe Negative*

## 2.3 Organizzazione del dataset

Il dataset è stato fornito in formato CSV, con due colonne principali:

- **Tweet**. Contenente il testo del tweet
- **Sentiment**. Contenente l'etichetta

Durante la fase di pre-processing, è stata aggiunta una terza colonna "cleaned_text", contenente la versione pulita del testo. La suddivisione in **training set (80%)** e **test set (20%)** è stata effettuata mantenendo la distribuzione originale delle classi tramite **stratificazione**.

# 3 Metodi utilizzati

## 3.1 Pre-processing del testo

Prima di addestrare il modello di Machine Learning per la Sentiment Analysis, è essenziale eseguire una fase di pre-processing sui dati testuali, al fine di ridurre la variabilità lessicale e rimuovere il rumore. Il dataset di partenza era costituito da tweet informali, spesso contenenti abbreviazioni, emoji, menzioni, hashtag, URL e simboli non rilevanti al fine dell'analisi del sentiment. Per questo motivo, ogni messaggio è stato sottoposto a una serie di trasformazioni.

Le operazioni di pre-processing hanno incluso i seguenti passaggi principali:

- Conversione in minuscolo
- Rimozione di URL, menzioni e hashtag
- Normalizzazione dei caratteri ripetuti
- Pulizia dei caratteri speciali
- Gestione delle emoji

Al termine di queste operazioni è stata creata una nuova colonna "cleaned_text" contenente le versioni normalizzate dei tweet originali.

## 3.2 Suddivisione del dataset

Dopo la fase di pulizia, il dataset è stato suddiviso in due sottoinsiemi distinti, uno destinato all'addestramento del modello (training set) e uno riservato alla sua valutazione (test set). La suddivisione è stata effettuata adottando una proporzione dell'80% per il training set e 20% per il test set in modo stratificato.

Il dataset risultante è stato così diviso:

- **Training set**: 16.757 tweet
- **Test set**: 4.190 tweet

## 3.3 Data augmentation e bilanciamento delle classi

Per migliorare affrontare lo sbilanciamento tra le classi nel training set, è stata adottata una tecnica di **data augmentation testuale**. Nello specifico, l'obiettivo è stato portare ciascuna classe del training set a contenere un numero uniforme di esempi, generando nuovi campioni artificiali per le classi meno rappresentate.

L'augmenter selezionato è stato "**SynonymAug**", che utilizza WordNet per sostituire alcune parole del testo originale con dei **sinonimi semanticamente coerenti**. Questa tecnica consente di generare frasi nuove e realistiche mantenendo il significato del tweet. Il risultato finale è un **training set quasi perfettamente bilanciato**, composto da circa 9.000 esempi per ciascuna classe.

## 3.4 Costruzione della matrice TF-IDF

Una volta ottenuto il dataset pulito e bilanciato, è stato necessario convertire il testo in una rappresentazione numerica compatibile con l'input di una rete neurale. Per farlo, è stato utilizzato il metodo **TF-IDF** (Term Frequency – Inverse Document Frequency), una tecnica per rappresentare i testi in forma vettoriale, che assegna a ciascun termine un peso proporzionale alla sua rilevanza all'interno del documento.

Il risultato è stata una matrice sparsa di dimensione 27.000x5.000 per il training set e una di dimensione 4.190x5.000 per il test set.

## 3.5 Il modello neurale

Per la classificazione è stato sviluppato un modello di **rete neurale** semplice, adatto per lavorare con vettori TF-IDF. L'architettura adottata è un classificatore feedforward, composto da pochi strati densi con attivazioni non lineari, normalizzazione e tecniche di regolarizzazione.

Nel dettaglio, il modello si basa sulla seguente sequenza di strati:

- **Input layer**. Riceve vettori di dimensione 5.000

- **Hidden layer**. Uno strato completamente connesso da 5.000 a 64 neuroni
- **Batch Normalization**. Normalizza l'output dello strato nascosto
- **Attivazione ReLU**. Introduce non linearità
- **Dropout (0.4)**. Disattiva in modo casuale il 40% dei neuroni durante l'addestramento, riducendo l'overfitting
- **Output Layer**. Uno strato lineare da 64 neuroni a 3 neuroni (uno per ogni classe)

## 3.6 Addestramento del modello

Per la fase di apprendimento è stata impiegata la funzione di perdita Cross-Entopy. L'algoritmo di ottimizzazione usato è stato **Adam**, un metodo di discesa del gradiente, con i iperparametro **learning rate** impostato a 0.001.

Per evitare overfitting, è stato adottato un meccanismo di **early stopping**, che interrompe l'allenamento se non si osservano miglioramenti significativi dopo 10 epoche. L'addestramento è stato eseguito per un massimo di **150 epoche**, ma il meccanismo di early stopping ha in media interrotto il training intorno alle prime 90. Durante l'allentamento sono state monitorate le metriche di accuratezza e di loss su entrambi i set, i cui andamenti sono stati visualizzato tramite TensorBoard.

# 4 Valutazione del modello

Al termine dell'addestramento, il modello è stato valutato in modo approfondito utilizzando diverse metriche standard per la classificazione multi-classe.

Una prima valutazione qualitativa è stata effettuata tramite la **matrice di confusione**, che evidenzia la distribuzione delle predizioni corrette ed errate per ciascuna classe.

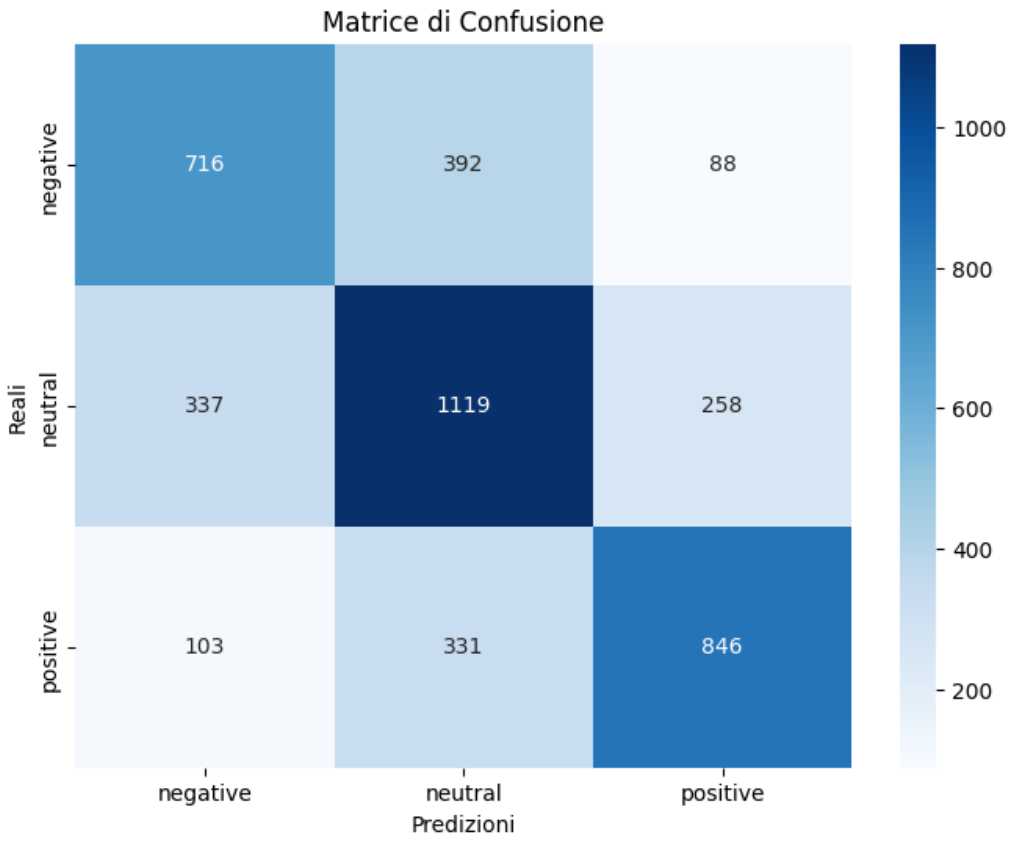



*Figura 6 - Matrice di confusione del modello sul test set*

La matrice mostra che il modello è generalmente accurato nel riconoscere i tweet **positivi** e **negativi**, mentre la classe **neutrale** tende ad essere leggermente meno distinta.

Per una valutazione quantitativa, sono state calcolate le classiche metriche di **Accuracy**, **Precision**, **Recall** e **F1-score**. I risultati, in media, sono i seguenti:

| Accuracy | Precision | Recall | F1-score |
|---|---|---|---|
| 0.6399 | 0.6421 | 0.6399 | 0.6403 |

La **curva ROC** (Receiver Operating Characteristich) è stata tracciata per ciascuna classe, visualizzando il rapporto tra **True Positive Rate** (TPR) e **False Positive Rate** (FPR).

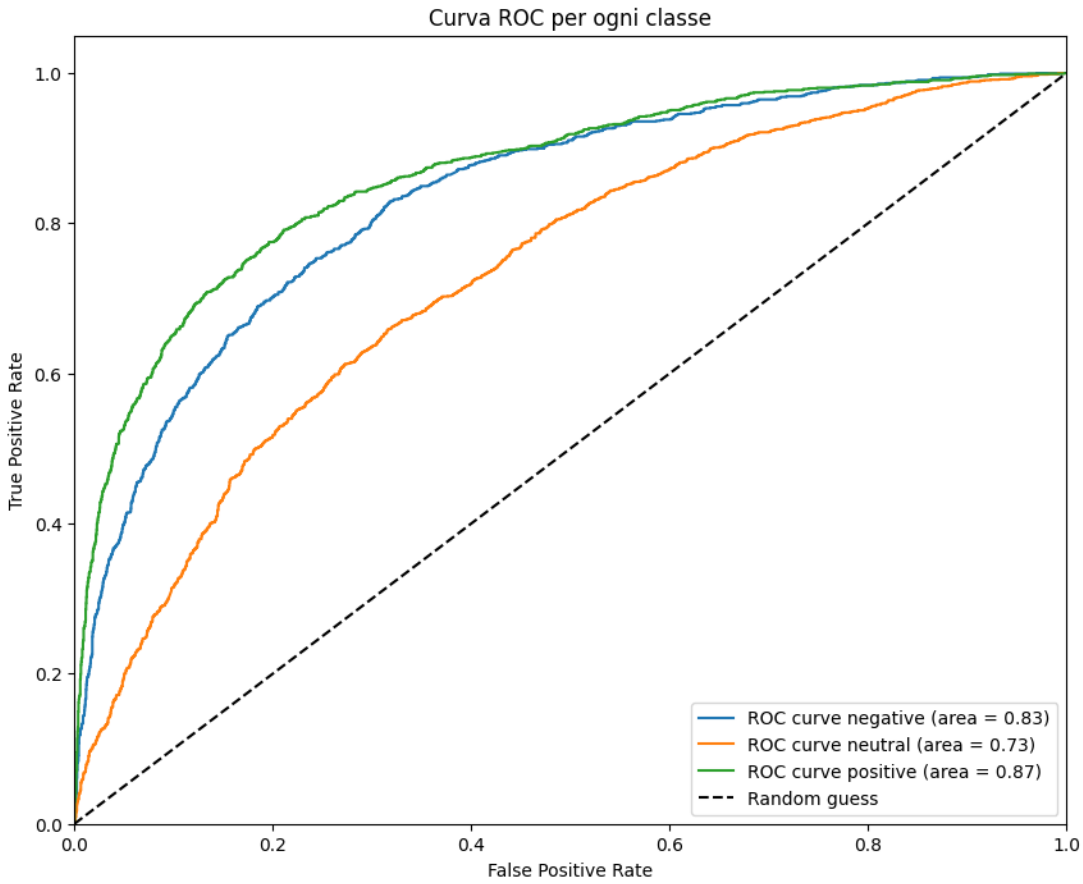



*Figura 7 - Curve ROC per ciascuna classe*

Infine, è stata visualizzata anche la **curva Precision-Recall**. Anche in questo caso, le curve evidenziano un comportamento stabile, in particolare per la classe positiva, mentre per la classe neutrale la precisione tende a calare con l'aumentare del recall, coerentemente con quanto osservato nella matrice di confusione.

# 5 La Demo: una applicazione reale del modello

Per dimostrare l'utilizzo pratico del modello sviluppato, è stato realizzato un semplice script Python chiamato "**sentiment.py**", che implementa una interfaccia a riga di comando per una comunicazione diretta con il modello. Questa Demo una applicazione concreta del modello e consente all'utente di inserire manualmente una frase da tastiera, ottenendo come risposta la relativa classificazione emotiva (positiva, neutrale o negativa).

## 5.1 Collegamenti con il notebook

Per garantire la riproducibilità del modello nello script, senza la necessità di addestrarlo da zero, sono stati salvati tre oggetti fondamentali al termine del Notebook contenente l'intero processo di addestramento del modello:

- **Model.pth**. Contiene i pesi addestrati della rete neurale. Viene caricato per eseguire le predizioni
- **Vectorizer.pkl**. Rappresenta il vettorizzatore TF-IDF utilizzato durante l'addestramento. È necessario per trasformare le nuove frasi nello stesso spazio vettoriale
- **Laberl_encoder.pkl**. Si tratta dell'oggetto Laberl Encoder che consente di convertire gli indici numerici nelle etichette originali

Tutti questi oggetti vengono caricato all'avvio dello script permettendo al modello di mantenere la coerenza con i dati su cui è stato addestrato.

## 5.2 Funzionamento della Demo

Il funzionamento si compone delle seguenti fasi:

1. L'utente esegue lo script da terminale con il comando

```
python sentiment.py
```

2. Dopo l’avvio, viene chiesto all’utente di inserire una frase
3. Il testo viene pre-processato nello stesso modo utilizzato durante la fase di addestramento
4. La frase viene trasformata in un vettore TF-IDF e passata in input al modello
5. Il modello restituisce la classe predetta insieme alla probabilità associata

Il ciclo è ripetuto fino all’interruzione manuale da parte dell’utente (CTRL+C).

A supporto, è stato anche fornito una **breve video dimostrativo** che mostra l’intero processo di utilizzo dello script.

# Conclusione

Il progetto svolto ha avuto come obiettivo lo sviluppo di un sistema in grado di classificare il sentiment di brevi testi utilizzando tecniche di pre-processing testuale e Machine Learning attraverso un modello di rete neurale semplice. Dopo una prima fase esplorativa condotta su un dataset di dimensioni ridotte, si è scelto di estendere il lavoro utilizzando un dataset più ampio, integrando inoltre tecniche per bilanciare le classi e aumentare il numero di esempi.

Lo sviluppo ha permesso di acquisire una visione più concreta e reale dell'intero processo di costruzione di un modello di Machine Learning, partendo dall'analisi esplorativa dei dati fino alla realizzazione di una applicazione reale e funzionante. È emersa, inoltre, **l'importanza della qualità e varietà del dataset**: un insieme di dati troppo semplice può infatti generare una falsa illusione di accuratezza e buone performance del modello.

Oltre alla fase di processamento dei dati, anche la progettazione della rete neurale (seppur semplice), ha richiesto attenzione nella **scelta dell'architettura e dei parametri** da utilizzare per evitare problemi di overfitting o underfitting. Infine, la possibilità di testare concretamente il modello su nuovi input testuali, ha reso più pratico e tangibile il lavoro svolto, dimostrando come un semplice progetto accademico possa essere applicato anche in contesti reali.

# Indice delle figure

Figura 1 - Distribuzione iniziale delle etichette nel dataset ..... 4
Figura 2 - Lunghezza media dei tweet per classe ..... 5
Figura 3 - Word Cloud classe Positive ..... 6
Figura 4 - Word Cloud classe Neutral ..... 6
Figura 5 - Word Cloud classe Negative ..... 7
Figura 6 - Matrice di confusione del modello sul test set ..... 11
Figura 7 - Curve ROC per ciascuna classe ..... 12